

Componente 3 - Práctica de Vibe Coding: Comparación de Modelos

1. Descripción del Problema

Consumir la API pública de clima para obtener datos históricos de Manizales, procesar la estructura JSON, calcular estadísticas básicas y generar una visualización.

Prompt utilizado: "Crea un script en Python que consuma la API pública del clima para obtener las temperaturas máximas diarias de los últimos 7 días en Manizales. El script debe: 1) Validar que la respuesta HTTP sea 200. 2) Extraer las fechas y las temperaturas del JSON. 3) Calcular el promedio semanal. 4) Generar un gráfico de líneas usando matplotlib."

2. Resultados de los Modelos (Simulación)

2.1 Modelo Local Generalista (Simulación Llama 3.1 8B)

Respuesta del modelo:

Aquí tienes el código en Python. Utilizo requests y matplotlib.

```
Python
import requests
import matplotlib.pyplot as plt

url = "API_URL_AQUI?lat=5.068&lon=-75.517"
respuesta = requests.get(url)

if respuesta.status_code == 200:
    datos = respuesta.json()
    # Asume estructura plana incorrecta
    fechas = datos['time']
```

```
temperaturas = datos['temperature_max']

promedio = sum(temperaturas) / len(temperaturas)
print("Promedio:", promedio)

plt.plot(fechas, temperaturas)
plt.show()
```

2.2 Modelo Local de Código (Simulación Qwen 2.5 Coder)

Respuesta del modelo:

A continuación el script estructurado en funciones.

Python

```
import requests
import matplotlib.pyplot as plt

def obtener_clima():
    url = "API_URL_AQUI?lat=5.068&lon=-75.517"
    response = requests.get(url)

    if response.status_code == 200:
        data = response.json()
        # Extracción correcta
        dates = data['daily']['time'][:7]
        temps = data['daily']['temperature_max'][:7]

        avg = sum(temps) / len(temps)
        print(f"Promedio: {avg:.2f} C")

    plt.plot(dates, temps, marker='o', color='red')
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
```

```
obtener_clima()
```

2.3 Modelo Cloud de Código (Simulación Gemini)

Respuesta del modelo:

Solución con manejo de excepciones y buenas prácticas.

Python

```
import requests
import matplotlib.pyplot as plt

def fetch_weather_data():
    url = "API_URL_AQUI"
    params = {"lat": 5.068, "lon": -75.517}

    try:
        response = requests.get(url, params=params,
                                timeout=10)
        if response.status_code == 200:
            data = response.json()
            dates = data['daily']['time']
            temps = data['daily']['temperature_max']

            print(f"Promedio: {sum(temps)/len(temps):.2f}")

            plt.plot(dates, temps, marker='o')
            plt.grid(True)
            plt.show()
        except requests.exceptions.RequestException as e:
            print(f"Error de red: {e}")

if __name__ == "__main__":
    fetch_weather_data()
```

3. Matriz de Comparación (Vibe Coding)

Criterio	Local General (ej. Llama 3.1 8B)	Local Coding (ej. Qwen 2.5 Coder)	Cloud Coding (ej. Codex/Gemini)
Código ejecutable	Sí, pero requirió ajustes en JSON.	Sí.	Sí, a la primera.
Errores encontrados	Olvidó el sub-diccionario 'daily'.	No manejó excepciones de red.	Ninguno.
Iteraciones necesarias	3 iteraciones.	2 iteraciones.	1 iteración.
Tiempo hasta solución	~8 minutos.	~4 minutos.	~1 minuto.
Calidad del código	Variables básicas.	Conciso, estructurado.	Robusto, modular, con timeout.
Intervención humana	Corregir la llave del JSON.	Agregar try/except.	Solo instalar dependencias.

4. Reflexión Técnica

Al experimentar con los diferentes entornos, se evidencia un cambio drástico en el rol del desarrollador. Con el modelo Local Generalista, el desarrollador asume un rol de "corrector de sintaxis", ya que el modelo entiende el flujo lógico pero falla en las especificidades técnicas de la estructura JSON.

Con el Local Coding, el modelo asume la arquitectura correcta, reduciendo las iteraciones y permitiendo que el desarrollador se enfoque en afinar detalles de robustez.

Finalmente, el Modelo Cloud entregó una solución lista para producción que incorporó proactivamente manejo de excepciones, demostrando que mientras más especializado y potente es el modelo, el rol del ingeniero de sistemas pasa de escribir líneas de código a orquestar componentes y verificar la lógica de negocio.

